SAKURA internet

Building a Web App with AI Engine

SAKURA AI Engine Practical Exercise Module4

https://www.sakura.ad.jp/

DATE
2026.9

COMPANY
SAKURA internet Inc.

SERVICE
SAKURA AI Engine

VERSION
Ver.1.0

Today’s Agenda

SAKURA AI Engine Hands-On consists of four sessions, 1 through 4.

No.1	**SAKURA AI Engine Hands-On 1** Log in to AI Engine, select a model, issue an account token, run inference, and build a RAG chatbot fed with a document	
No.2	**SAKURA AI Engine Hands-On 2** A setup that uses AI to automate everything from meeting-audio transcription to summarization and minutes	
No.3	**SAKURA AI Engine Hands-On 3** Combining image and text input to design AI output that supports business decisions	
No.4	**SAKURA AI Engine Hands-On 4** Embedding image/text AI calls into a Web app, all the way to running it in the browser	**★ You are here**

Goal of This Class

Use the distributed code to understand how a Web app built with AI Engine works.

What We will Do

1. Set up the runtime environment
2. Read through the code’s flow
3. Fill in a few blanks in the code
4. Run it, ask a question, and check the answer and its sources

The App We will Build

- Send a natural-language question against the registered RAG data.
- View the result in your browser

- Change top_k / threshold from the screen
- Display the answer and its sources (name, chunk, distance, content)

Turn the RAG you ran with curl into something you can use from a browser

Overview 1

The Environment We'll Build

WSL + Python Virtual Env (venv) + Flask

The System We'll Build in This Class

The user is on the left, AI Engine on the right. The question goes out, and the answer comes back.

User Side (Your PC)

Browser

Screen / JavaScript

Flask

Build and send the request

Receive and return the response

★ Holds the token (never exposed to browser/JS)

AI Engine Side (SAKURA Cloud)

AI Engine (documents/chat)

① Find chunks close to the question (`e5-large/cosine`)

② Get the top top_k results whose distance is threshold or less

③ Generate the answer based on them (`gpt-oss-120b`)

Reference

RAG Documents (already embedded)

Outbound

Question **"Which Thai restaurant is the most popular in Thailand?"** `top_k = 3` `threshold = 0.5`

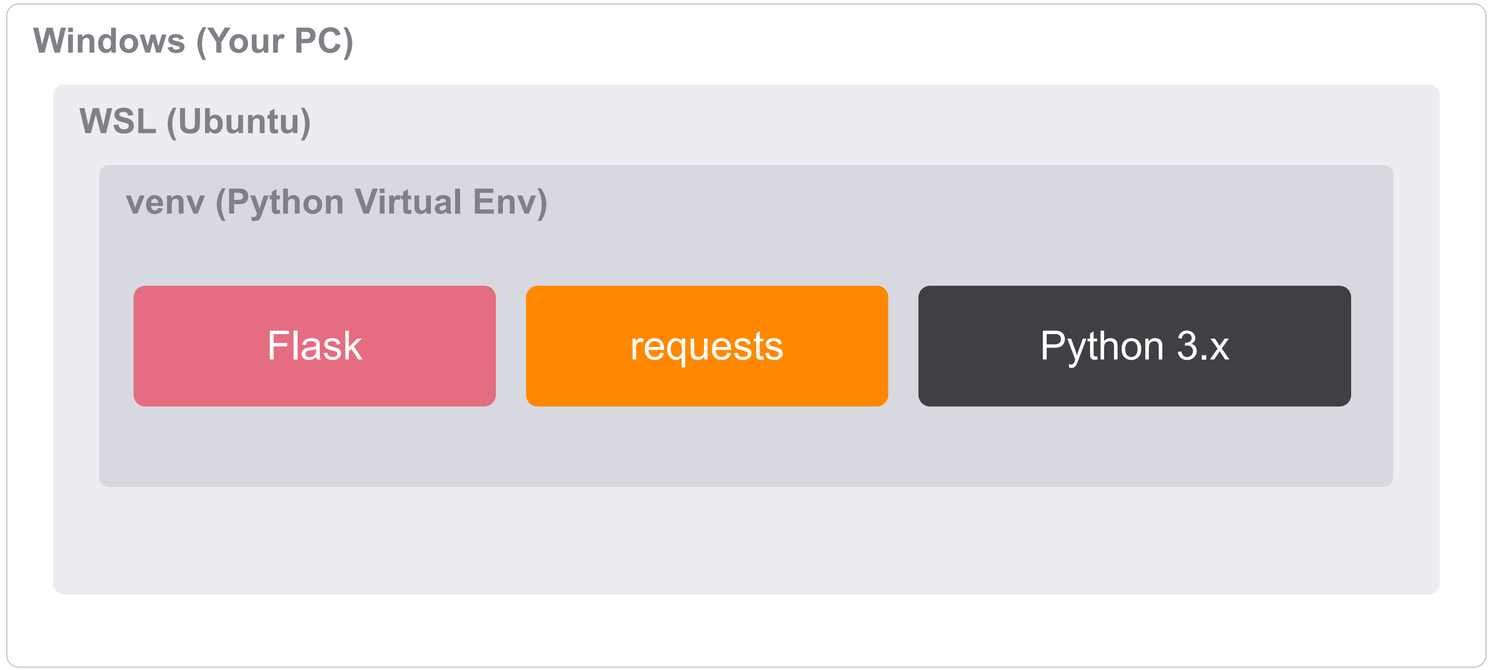
Inbound

Answer **"According to the registered documents, ●● and XX are popular in Bangkok."**

Sources = [document name / chunk number / `distance` / content] for the top `top_k` results

Environment Overview

Set up a Windows + Linux, and inside that a Python virtual env, in your PC.



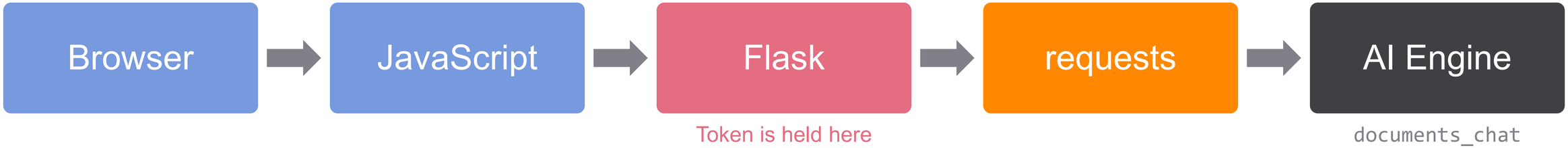
- **Windows** Your PC
- **WSL** Run the class commands here Linux
- **venv** Dedicated to Python env
- **Flask** Web server
- **requests** API calls

Set up your own Windows + Linux + Python environment for this class.

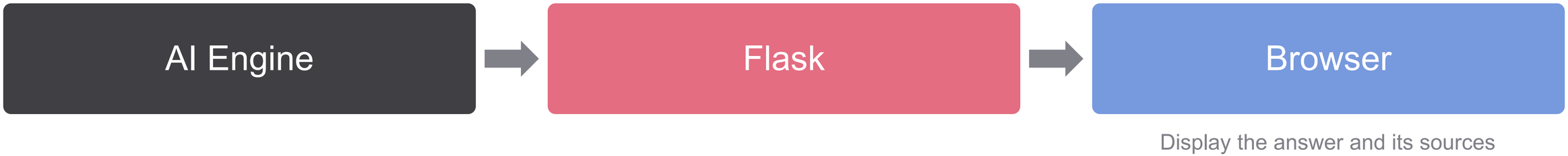
Overall Structure (Data Flow)

Until now you called curl directly with AI Engine; now call it through the browser and Flask.

Outbound (Request)



Reply (Inbound Response)



The account token is held only by Flask; it stays hidden from the browser and JavaScript.

Why Flask Is Needed

Browser and AI Engine need a Web server in between. That’s Flask.

Entry Point

The window that receives requests from the browser (routing).

Mediates Between Browser and AI Engine

“requests” calls AI Engine and formats the result before returning it.

Hides the Token

Holds the account token on the backend, never exposing it to the browser.

A one-off curl isn’t enough —Flask is needed to turn this into a Web app the screen can call any number of times.

What Is venv?

A separate, independent Python environment. Also called a virtual environment.

Why Separate Environments?

- Avoids conflicts between dependencies
- Doesn’t affect other projects
- Installs only what you need
- Easy to reproduce the environment

Picture It

Install everything system-wide

Versions clash easily

venv separates things into boxes

Independent and safe, per project

This time we’ll create a dedicated box with only Flask and requests installed.

Directory Structure

Extract the distributed zip and lay it out like this.

```
04_rag-web-app/
├── app.py
├── requirements.txt
├── README.md
├── templates/
│   └── index.html
└── static/
    ├── app.js
    └── style.css
```

`templates/`

Where Flask's returned HTML lives

`static/`

Where the JS/CSS the browser reads lives

Flask auto-loads `templates` and `static` from fixed locations, so it must be laid out this way.

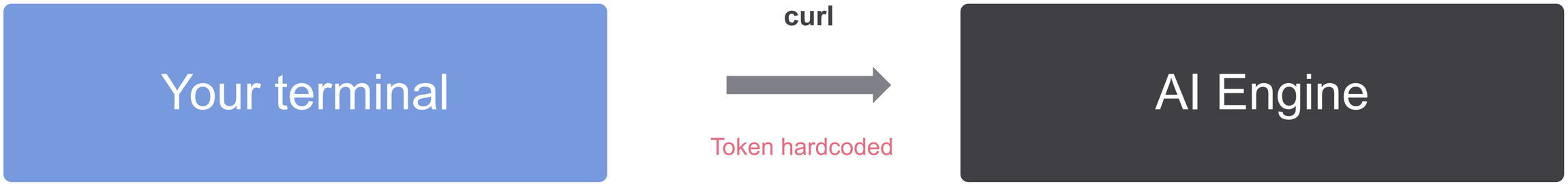
Overview 2

curl vs. Turning It Into a Web App

From calling curl directly, to an app with a screen.

How We’ve Done It So Far

In sessions 1–3, you used curl from the terminal to call AI Engine directly.



- Run commands by hand
- Only you run it
- Result shows in the terminal
- One-off calls, one at a time

It works, but it’s all self-contained inside your own terminal.

Web App: The Challenge

curl is for things like checking behavior works; it’s not something anyone can just use.

A Command Every Time

Users must remember the terminal and the command.

The Token Is Exposed

The token rides along in the command. Share it, and it leaks.

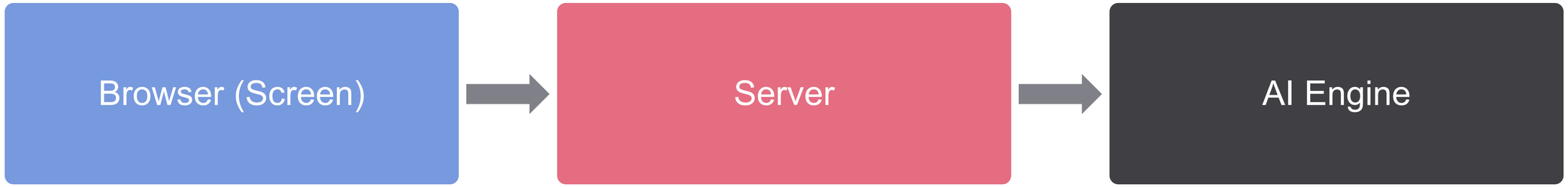
No Screen

No input field, no result display — you can’t hand it to someone. You’d have to read the code and parse the result — a hassle

From “I run it myself” to “someone else can use it”.

What It Means to Turn This Into a Web App

A screen instead of a command; backend processing instead of manual steps.

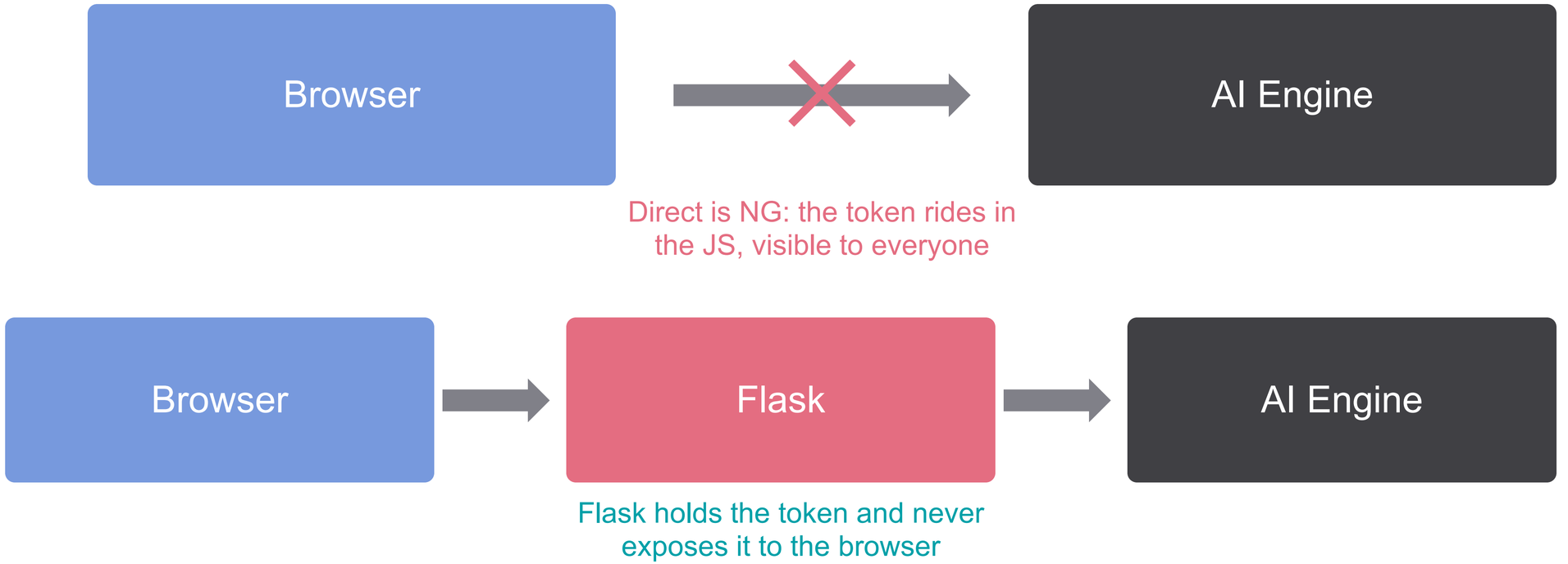


With curl		With the Web App
Type a command	➡	Click a button on screen
Shows in the terminal	➡	Formatted and shown on screen
Token on hand	➡	Hidden inside the server

By operating the browser, the user can trigger a call to AI Engine.

Why a Server Needs to Sit in Between

Anyone can view the browser's code, so secrets belong on the server side.



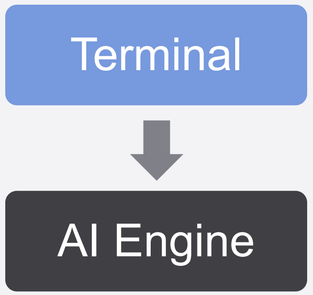
- Processing that holds a secret (the token) belongs on the server
- The server also handles input validation and result formatting

That's why a server in between —Flask — is needed.

Before / After

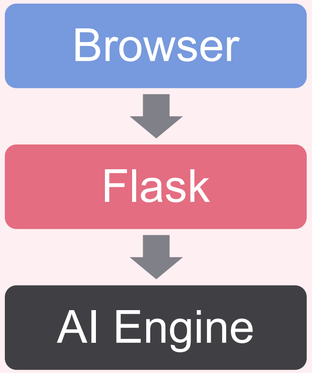
The task is the same. how you call it changes.

Before: Invoking via the curl command



Token = on hand

After: Web App



Token = inside the server

Action: command → screen | Display: terminal → screen | Token: on hand → server

Who you call (AI Engine) is the same. What changes is which endpoint and how you call the API.

Overview 3

Environment Setup

WSL assumes it’s already installed. From here, you’ll actually start working hands-on.

Setup Flow

We’ll proceed in 4 steps.

1. Place the distributed files in your WSL home
2. Rearrange into the correct folder structure
3. Create and enter a virtual environment (venv)
4. Install packages and set the token

Work through them in order, and let’s launch the Web app!

Inside the Distributed Files

Github: download the data. Let’s check what each file does.

https://github.com/onigiribouya/sakura-ai-engine-module4-download-zip/raw/refs/heads/main/module4-src.zip

app.py	The Flask server itself. Receives requests, validates, calls requests, formats results
index.html	The screen’s skeleton (HTML)
app.js	Gets input, sends fetch requests, shows results (frontend logic)
style.css	The screen’s look (styling)
requirements.txt	List of required packages (Flask, requests)
README.md	Usage instructions

These 6 files complete the round trip: browser → Flask → AI Engine.

The Folder Structure We’re Aiming For

Before you download anything, here’s the folder shape we’re building toward.

Target Folder Structure

```
04_rag-web-app /
├── app.py
├── requirements.txt
├── README.md
├── templates/
│   └── index.html
└── static/
    ├── app.js
    └── style.css
```

Keep this shape in mind — first, let’s download the files.

WINDOWS EXPLORER | NOT IN WSL

Download the Distributed Files

Download the exercise files from GitHub. No Git commands needed.

```
Download link:
https://github.com/onigiribouya/sakura-ai-engine-module4-download-zip/raw/refs/heads/main/module4-src.zip
```

- Clicking this link downloads `module4-src.zip` directly. No need to visit the repository page first.
- The file is saved to your Windows Downloads folder.
- This step uses only your browser — no WSL terminal needed yet.

Now module4-src.zip is in your Downloads folder, ready to extract.

WINDOWS EXPLORER | NOT IN WSL

Extract It Into Your WSL Folder

Extract the ZIP directly into WSL, so there's no need to copy it over afterward.

! DO THIS IN WINDOWS

1. Open `¥¥wsl.localhost¥Ubuntu¥home¥<your username>¥` in Explorer
2. Right-click **module4-src.zip** (in Downloads) → Extract All...
3. In the extraction dialog, set the destination to the folder above
4. Rename the extracted folder from **module4-src** to **04_rag-web-app**

- The extracted folder is normally named `module4-src`. Rename it so the later commands and slides match.
- How to check it worked: run ls ~ in WSL — 04_rag-web-app should appear in the list.
- **Everything so far (download + extract) used only Explorer. Next, switch back to your WSL terminal (the black window) for the remaining steps.**

Now ~/04_rag-web-app inside WSL has all the distributed files, laid out correctly for Flask.

Set Up the Correct Folder Structure

This is the shape from earlier — let's check what you actually have now, then fix it in the terminal.

Right After Extracting (zip layout as-is)

```
04_rag-web-app /
├── app.py
├── requirements.txt
├── README.md
├── index.html
├── app.js
└── style.css
```

Correct Layout

```
04_rag-web-app /
├── app.py
├── requirements.txt
├── README.md
├── templates/
│   └── index.html
└── static/
    ├── app.js
    └── style.css
```

`index.html` goes to `templates`, and `app.js` and `style.css` go to `static`.

Create Folders and Move Files

Switch back to your WSL terminal (the black window) and go into 04_rag-web-app.

```
$ cd ~/04_rag-web-app           # go back into the directory
$ mkdir templates static        # create 2 directorys
$ mv index.html templates/      # move HTML into templates
$ mv app.js style.css static/   # move JS/CSS into static
```

- After moving, check the layout with ls and ls templates static.
- app.py · requirements.txt · README.md stay right where they are, at the top level.

That completes the correct folder structure.

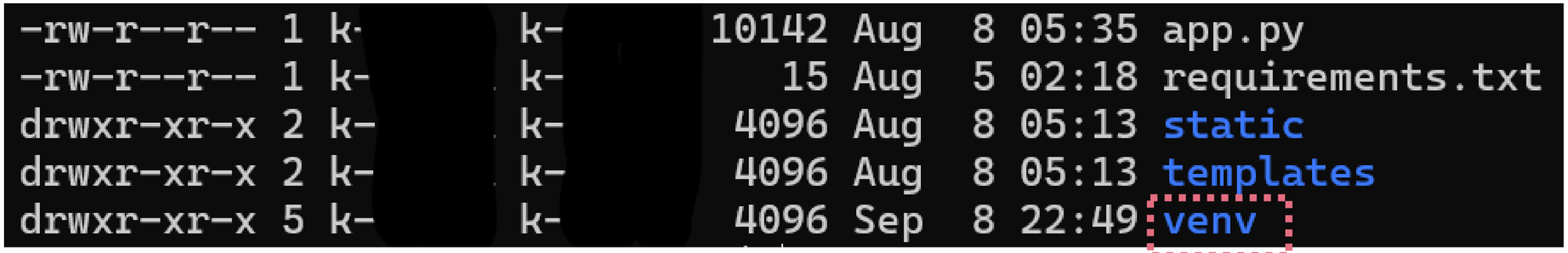
Create a Virtual Environment (venv)

`04_rag-web-app` : create a dedicated Python environment inside it.

```
$ python3 -m venv venv
```

- Inside `04_rag-web-app`, a folder called `venv` (your dedicated environment) is created.
- On WSL (Ubuntu), use `python3` instead of `python`.
- You only create it once; no need to redo it after that.
- How to check it worked: run ls — you should see a new **venv** folder.

Example: ls -l output after venv is created



At this point, you haven't entered the environment yet. The next step enters it.

Entering / Leaving venv

The sign to watch for is **(venv)** at the start of the prompt.
With (venv) showing, you need to run Flask.

Entering the virtual env (activate)

```
user@host:~/04_rag-web-app$ source venv/bin/activate

(venv) user@host:~/04_rag-web-app$
```

At the front, **(venv)** once it shows, you're in the virtual environment.

Leaving (deactivate)

```
(venv) user@host:~/ 04_rag-web-app$ deactivate

user@host:~/04_rag-web-app $
```

At the front, **(venv)** once it's gone, you've left the virtual environment.

From here on, do all work with (venv) showing.

Install the Packages

`(venv)` is showing before running this.

```
(venv) $ pip install -r requirements.txt
```

- `requirements.txt` lists Flask and requests, which get installed.
- `(venv)` If you run this without showing, it installs elsewhere. Be sure to check.
- You can check whether it's installed with `pip list`.
    - Look for Flask and requests in the list — the other packages are their dependencies, installed automatically.

Now the environment to run the app is ready!

Set the Token

Set the account token as an environment variable.

```
(venv) $ export AI_ENGINE_TOKEN="<your token>"
```

- Flask reads the token from this environment variable. The browser and JS never get to see the token's contents.
- This setting disappears when you close that terminal. If you reopen it, set it again.
- You can check whether it's set with **`echo $AI_ENGINE_TOKEN.`**

Never write the token directly in code — read it from an environment variable.

Setup Complete Checklist

You're ready once these 4 are all true.

- ✓ `04_rag-web-app` has `templates/` and `static/` laid out correctly
- ✓ shows `(venv)` at the start of the prompt
- ✓ `pip list` has Flask and requests
- ✓ `echo $AI_ENGINE_TOKEN` shows the token

Next, move on to filling in the code and running it!

Overview 4

Filling In the Code

Think about where in the code the values you typed with curl should go

The Big Picture of the Blanks

3 groups, 7 blanks in total.**Build → Send → Receive** — that's the order we'll follow.

1. **Build what to send** app.py build_payload (`__(1)__ __(2)__ __(3)__`)
2. **Send what you built** app.py requests.post (`__(4)__ __(5)__`)
3. **Retrieve the result that comes back** app.js (`data.__(1)__ data.__(2)__`)

Build the data to send, send it to AI Engine, receive the data that comes back, and display it.Think it through in this order.

① Build What to Send

Put the values you passed via `--data` into `payload` below.

app.py build_payload (fill in the blanks)

```
def build_payload(query, top_k, threshold):
    payload = {
        "model": EMBEDDING_MODEL,
        "chat_model": CHAT_MODEL,
        "query": __(1)__,
        "top_k": __(2)__,
        "threshold": __(3)__,
        "distance_type": DISTANCE_TYPE,
    }
```

The --data you saw with curl

```
--data '{
  "query": "...",
  "top_k": 3,
  "threshold": 0.5
}'
```

What goes in?

- `model` · `chat_model` · `distance_type` are fixed constants. They aren't blanks.

The 3 values from curl's `--data`. What goes in each one?

① Answer: Try Plugging In Values First

Here’s what it looks like if you write the values directly, the way they were set on the screen.

app.py build_payload (with values filled in)

```
payload = {
    "model": EMBEDDING_MODEL,
    "chat_model": CHAT_MODEL,
    “query”: “Which Thai restaurant is best?",
    "top_k": 3,
    "threshold": 0.5,
    "distance_type": DISTANCE_TYPE,
}
```

Screen Input (example)

Question

Can I return this?

top_k

3

threshold

0.5

As is, the value is fixed: whatever you type on the screen won’t be reflected.

① Answer: It’s Actually in Variables

The values entered on the screen are in the variables **`query`** / **`top_k`** / **`threshold`**.

app.py build_payload (answer)

```
payload = {
    "model": EMBEDDING_MODEL,
    "chat_model": CHAT_MODEL,
    "query": query,
    "top_k": top_k,
    "threshold": threshold,
    "distance_type": DISTANCE_TYPE,
}
```

Screen Input	→ Variable
The screen’s “Question”	`query`
The screen’s “Number of References”	`top_k`
The screen’s “Threshold”	`threshold`

- The variable holds whatever value was entered on the screen at that time. In other words, writing the variable name means the input goes straight into payload.

Assign the variable, not a fixed value. The value entered on the screen goes straight to AI Engine.

1 Answer (Final Version)

Just take the value received as an argument and put it straight into payload.

app.py build_payload (answer)

```
    payload = {
        "model": EMBEDDING_MODEL,
        "chat_model": CHAT_MODEL,
        "query": query,
        "top_k": top_k,
        "threshold": threshold,
        "distance_type": DISTANCE_TYPE,
    }
```

__(1)__ **query**
The question text (validated string)

__(2)__ **top_k**
Maximum number of documents to retrieve

__(3)__ **threshold**
Vector-distance threshold (0–2)

The same 3 from curl's `--data`. Even the names stay the same.

2 Send What You Built

Where do curl's destination URL and `--data` go inside `requests.post`?

app.py requests.post (fill in the blanks)

```
response = requests.post(
    __(4)__,
    headers=headers,
    json=__(5)__,
    timeout=REQUEST_TIMEOUT,
)
```

What it looked like with curl

```
curl <destination URL> ¥
  --header "..." ¥
  --data '{...}'
```

What goes in?

- URL → 1st argument, `--data` → `json=`. That's the mapping.

Map curl's URL and `--data`. In requests, that's the 1st argument and `json=`.

2 Answer

Send the payload you built in 1 , here.

app.py requests.post (answer)

```
response = requests.post(
    AI_ENGINE_URL,
    headers=headers,
    json=payload,
    timeout=REQUEST_TIMEOUT,
)
```

__(4)__ AI_ENGINE_URL
Destination (the constant defined above)

__(5)__ payload
The body you built in 1

- curl's URL is the 1st argument, `--data` is `json=`.

Send the payload you built in 1, here.

3 Retrieve the Result That Comes Back

Pull the answer and its sources out of the JSON that comes back from Flask.

Located in static/

app.js Extracting the response (fill in the blanks)

```
// 7. Display the answer and its sources
showAnswer(data.__(1)__);
showSources(data.__(2)__);
```

- Path: static/app.js

Shape of the JSON that comes back

```
{
  "answer": "...",
  "sources": [ {...}, {...} ]
}
```

What goes in?

- Write the JSON key name directly after the `.` dot

Which key in the returned JSON is the answer, and which is the sources?

3 Answer

Just specify the returned JSON’s key names, as they are.

app.js Extracting the response (answer)

```
// 7. Display the answer and its sources
showAnswer(data.answer);
showSources(data.sources);
```

`data.__(1)__ answer`

The answer text → to the screen

`data.__(2)__ sources`

The sources array → lists name, chunk number, distance, and content

- `sources` is an array; each item is listed on screen one by one.

`answer` and `sources` together give you the answer and its sources on screen.

Wrapping Up the Blanks

Once all 7 are filled in, save and move on.

- ✓ app.py build_payload: `query` / `top_k` / `threshold`
- ✓ app.py requests.post: `AI_ENGINE_URL` / `payload`
- ✓ app.js response: `data.answer` / `data.sources`

- Don’t forget to save. A missed blank causes an error on the next run.

Once you’re done, on to 6 Run It.

Overview 5

Run It

Start the server, ask it a question from the browser, and finally shut the server down.

Running Flow

From start to finish, we'll go through these 4 steps.

1. `(venv)` enter it (run `activate` if it's not showing)
2. `python app.py` to start the server
3. Open the displayed URL in your browser
4. Shut it down with `Ctrl + C`

Start it up, use it, then shut it down. That's one full cycle.

Start the Server

`(venv)`: check the token, then start it.

```
(venv) $ echo $AI_ENGINE_TOKEN   # check the token
<OK if a token appears here>

(venv) $ python app.py           # start the server
```

- If `(venv)` isn't showing, enter it with `source venv/bin/activate`.
- If the token is empty, run `export AI_ENGINE_TOKEN=...` first.

Once you're ready, `python app.py` starts the server.

What You’ll See on Startup

Once it starts, the terminal shows something like this.

```
 * Serving Flask app 'app'
 * Debug mode: on
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

- `Running on ...` — that line is your app’s URL.
- It’s debug mode, so saving your code reloads it automatically.
- Press `CTRL+C to quit` — that’s how to stop it.

Open this URL in your browser, and the app is ready to use.

Why 127.0.0.1 : 5000?

Let’s break down what the displayed URL means.

`http://127.0.0.1:5000`

`127.0.0.1`

Stays inside your own PC. Never exposed externally or to the internet (safe for classroom use).

`5000`

Flask’s default dev-server port. The number that serves as the app’s entry point.

- Specified in `app.py`’s `app.run(host="127.0.0.1", port=5000)`.

A safe development server that only runs inside your own PC.

Open It in Your Browser

Open the displayed URL and actually ask it a question.

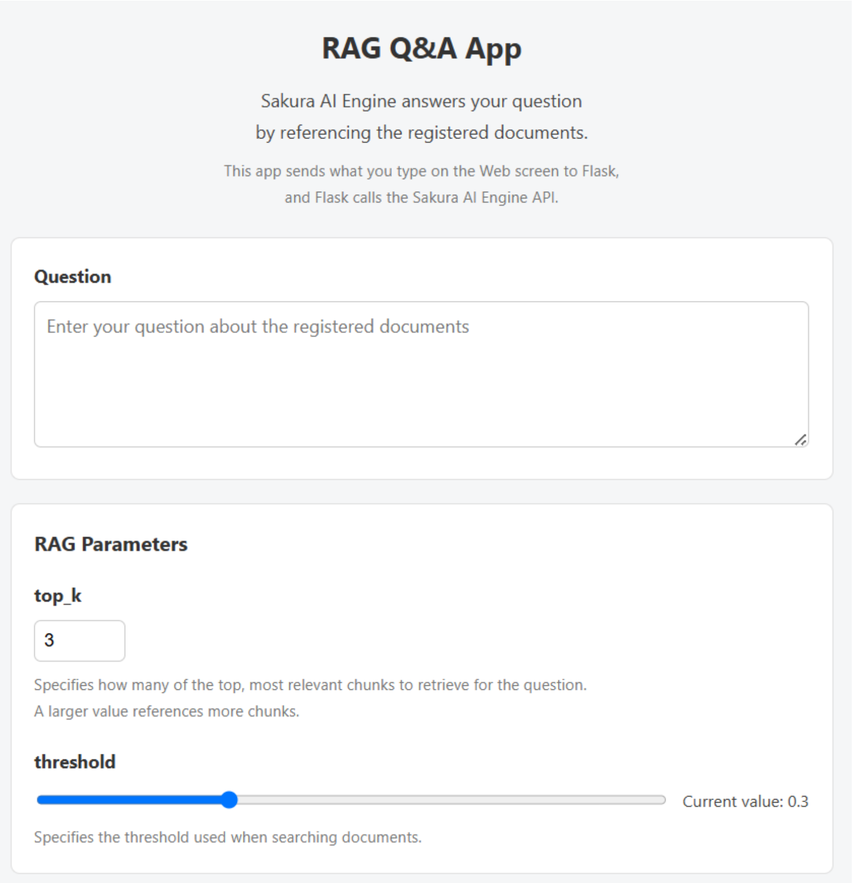
- In the terminal, **`http://127.0.0.1:5000`** — **Ctrl + click**

  (if it doesn’t become a link, type it into the browser’s address bar by hand)

- Once the page opens, type your question and set `top_k` / `threshold`.
- Press “Ask” and the answer and its sources appear.

  - ✓ Once the page opens, ask something about the documents you registered in Module 1 — and set top_k / threshold.
  - ✓ Tip: match your question's language to your registered documents for the best results.

Along with the answer, you can also check the sources (name, chunk number, distance, content).



Nice work! You just called AI Engine through your own Web app, start to finish.

Try It Out: What Happens If...?

Now that it's working, let's experiment a bit!

- Ask about something not in your registered documents

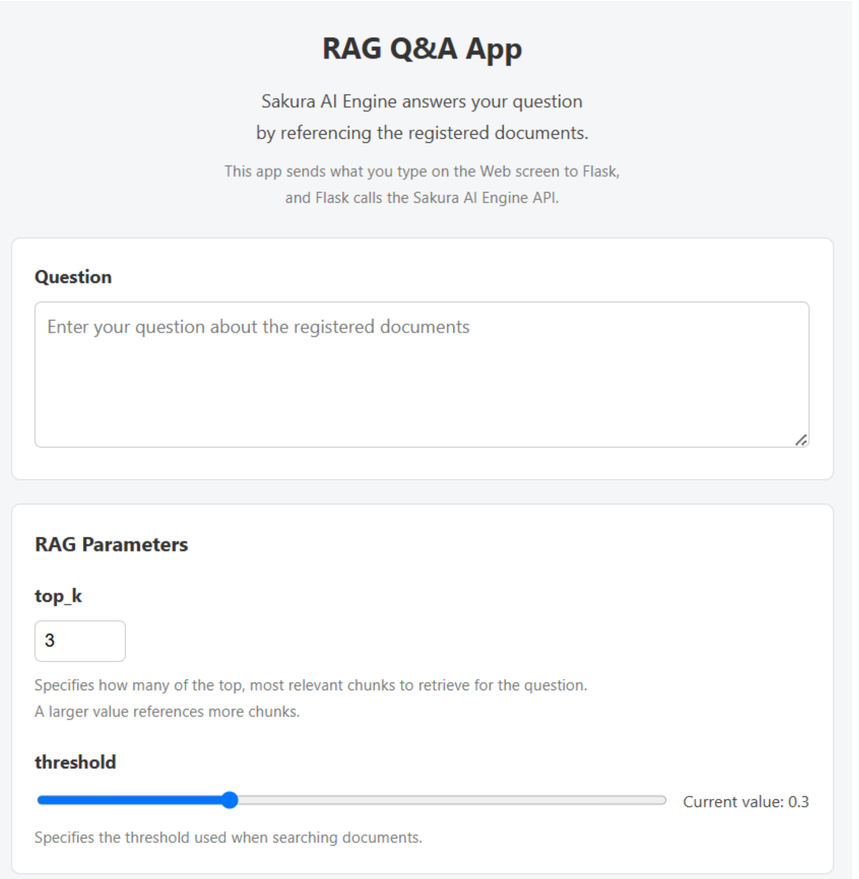
  What happens: the answer likely says it can't find relevant information — a good way to confirm the app is really searching your documents, not just guessing.

- Register a new document from the Sakura AI Engine Control Panel, then ask about it

  What happens: check whether the new document gets pulled in as a source — a live look at how documents_chat searches everything you've registered. registered.

- Ask in a different language than your registered documents

  What happens: search quality may vary — multilingual-e5-large supports cross-lingual search, but matching your question's language to your documents usually works best.



There's no single right answer here. The point is to see how top_k, threshold, and document language actually change what comes back.

Shut Down the Server

Once you're done, stop the server from the terminal.

```
(venv) $ python app.py
 * Running on http://127.0.0.1:5000
^C                          # press Ctrl + C
(venv) $
```

- Press `Ctrl + C` in the terminal to stop it. The prompt comes back.
- To run it again, use `python app.py`.
- To leave the virtual env, use `deactivate` (`(venv)` disappears).

Start with `python app.py`, stop with `Ctrl + C`. That's the basic operation.

Optional | For Those Who Finish Early: Extra Challenge 1

Swap the Answer-Generation Model

Switch to a different answer-generation model and compare how the answers differ.

```
CHAT_MODEL = "gpt-oss-120b"                # until now
        ↓
CHAT_MODEL = (try different models and see what works well)       # after (example)
```

Recommended Models

- `preview/gemma-4-31B-it`: Google's multilingual model.
  Its English is natural too, and the difference in style from `gpt-oss-120b` (general-purpose, larger) is easy to spot.

* All of these are public previews (may change or end without notice).
* Copy the model name exactly as shown in the model list / Playground, including the `preview/` prefix.

⚠ Never Change This

`EMBEDDING_MODEL`(`multilingual-e5-large`) **stays the same.**

Registered documents are already embedded with this model. Change the embedding model and search stops matching up. Only the answer-generation side (**`CHAT_MODEL`**) is safe to change.

IMPORTANT **Only change `CHAT_MODEL`. Never change the embedding model!**

Optional | For Those Who Finish Early: Extra Challeng 2

Change the Defaults and Logging

Make small changes to the screen and server and see how the behavior changes.

Change the Defaults / Range

- In `index.html`, change the default `top_k` / `threshold` values.
- To widen the upper limit, change `index.html`'s `max` slider together with `app.py`'s `TOP_K_MAX` (e.g. 10→20).

- Change the number of references or the threshold, and watch how the answer and sources shift.

`* Update both the screen (index.html)` and the server (`TOP_K_MAX`) together, or validation will reject it.

Add More Backend Logging

`app.py`: add more `print` calls to send data to the terminal.

```
print(f"sources={len(result['sources'])} items")
print(f"answer={result['answer'][:40]}")
return jsonify(result), 200
```

See what's happening on the server side, not just on screen.

Tweak both the screen and the server a bit, and see how the behavior differs.

Add a Feature to Read the Answer Aloud

It takes real work, but it’s doable with what you learned today plus a bit of extra knowledge.

Browser → (answer text) → **Flask** → (text) → **AI Engine text-to-speech** → (audio WAV) → **Flask** → (play) → **Browser**

1. `index.html`: add a “Read Aloud” button.
2. `app.js`: on click, send the currently displayed answer text to Flask’s new endpoint `/api/speak`.
3. `app.py`: add the route `/api/speak` and call AI Engine’s text-to-speech API with requests (token via env var again).
4. `app.py`: return the audio (WAV) that comes back to the browser.
5. `app.js`: play the received audio → save and re-run to check the read-aloud works.

Same as with RAG, “browser → Flask → AI Engine” — try building it this time for speech synthesis.

If you’re not confident with programming, you could even have AI Engine help you code the implementation…

Hints for the Read-Aloud Feature

OpenAI TTS-compatible. Send `model` · `voice` · `input` and get audio back.

app.py (new route / also import Response)

```
SPEECH_URL = "https://api.ai.sakura.ad.jp
             /v1/audio/speech"

@app.route("/api/speak", methods=["POST"])
def speak():
    text = request.get_json().get("text","")
    payload = {"model": "<speech model ID>",
               "voice": "<voice ID>", "input": text}
    headers = {"Authorization":
        f"Bearer {os.getenv(TOKEN_ENV_NAME)}"}
    r = requests.post(SPEECH_URL,
        headers=headers, json=payload, timeout=60)
    return Response(r.content,
        mimetype="audio/wav")
```

static/app.js (send the answer and play the audio)

```
speakBtn.addEventListener("click",
  async () => {
  const res = await fetch("/api/speak", {
    method: "POST",
    headers: {"Content-Type":
              "application/json"},
    body: JSON.stringify({
      text: answerText.textContent })
  });
  const blob = await res.blob();
  const url = URL.createObjectURL(blob);
  new Audio(url).play();
});
```

- Check the speech model ID and voice ID under “Available Speech Models” in the Control Panel (terms agreement required first).
  Example: `model=“kasukabetsumugi” / voice=“normal”` (here, Kasukabe Tsumugi’s voice). Other voices, like Zundamon, are available too.
- For the latest endpoint and parameter details, check AI Engine’s API documentation.

AI Engine’s text-to-speech API. Think about whether you can build it with the same flow as RAG.

Today’s Wrap-Up

Great work!! Let’s check off what you can now do.

- ✓ **curl used to call AI Engine — now you can call it from a Web app**
- ✓ **You experienced the flow: Browser → JavaScript → Flask → AI Engine.**
- ✓ **curl and requests — you learned to read the mapping between them**
- ✓ **You filled in the blanks in the code, ran it, and confirmed `top_k` / `threshold` and the sources**
- ✓ **You saw why the token stays hidden on the Flask side, and how it’s implemented**